2/5进度情况

任务三学习笔记

今天继续任务三的学习，主要是关于运行和理解CNN模型在CIFAR-10数据集上的训练代码，感觉学到了好多深度学习的基础知识！作为大一新生，有些内容对我来说挺有挑战的，但通过记笔记梳理思路，感觉清晰多了。以下是我对今天学习内容的详细总结，打算存到Git仓库里，记录自己的成长

学习内容总结

- **任务目标**：任务三的重点是学习和运行一个简单的CNN模型，训练它在CIFAR-10数据集上进行图像分类。今天成功运行了`cifar10_cnn.py`文件，完成了10个epoch的训练，并通过逐行解析代码和结果分析，理解了深度学习的基本流程。
- **关键知识点**：
  - **CNN模型结构**：今天详细学习了一个简单CNN模型的组成，包含2个卷积层（Conv2d）和3个全连接层（Linear）。卷积层用来提取图像特征，池化层（MaxPool2d）缩小特征图尺寸，全连接层进行分类。复习要点：
    - 卷积层：如`Conv2d(3, 16, 3)`，输入3通道（RGB），输出16个特征图，卷积核3x3。
    - 池化层：`MaxPool2d(2, 2)`，将特征图尺寸减半。
    - 前向传播：输入图像经过卷积、池化、展平，再通过全连接层输出10个类别的预测。
  - **数据预处理和加载**：学习了如何用`torchvision`加载CIFAR-10数据集，并通过`transforms`进行预处理。复习要点：
    - `ToTensor()`：图像转张量，范围[0,1]。
    - `Normalize()`：标准化数据，范围[-1,1]，有助于训练。
    - `DataLoader`：批量加载数据，`batch_size=64`，训练时打乱数据（`shuffle=True`）。
  - **训练过程**：理解了训练的核心步骤，包括前向传播、计算损失、反向传播和参数更新。复习要点：
    - 损失函数：`CrossEntropyLoss()`，用于多分类。
    - 优化器：`Adam`，学习率0.001，更新模型参数。
    - 训练循环：每个epoch遍历数据集，计算损失和准确率，每200个batch输出一次中间结果。
  - **结果分析**：训练了10个epoch，记录了训练损失、训练准确率和测试准确率的变化。复习时可以看这个表格：

    ```
    | Epoch | Train Loss | Train Acc (%) | Test Acc (%) |
    |-------|------------|---------------|--------------|
    | 1     | 0.273      | 49.02         | 60.36        |
    | 2     | 0.219      | 64.32         | 67.27        |
    | 3     | 0.187      | 70.99         | 68.90        |
    | 4     | 0.154      | 76.08         | 69.27        |
    | 5     | 0.128      | 80.82         | 71.15        |
    | 6     | 0.103      | 85.39         | 71.94        |
    | 7     | 0.079      | 89.36         | 72.18        |
    | 8     | 0.061      | 92.34         | 71.15        |
    | 9     | 0.043      | 94.67         | 70.74        |
    | 10    | 0.032      | 95.98         | 70.14        |
    ```

    分析要点：

        - 训练损失从0.273降到0.032，训练准确率从49.02%升到95.98%，模型在训练数据上表现很好。
        - 测试准确率最高72.18%（Epoch 7），最终70.14%，训练准确率远高于测试准确率，说明有过拟合现象。
- **实践操作**：
    - 成功运行了`cifar10_cnn.py`，完成了10个epoch训练，使用了GPU（`cuda`），速度很快。
    - 观察了训练过程中的损失和准确率变化，保存了模型文件`cifar10_cnn.pth`，还尝试理解了代码绘图部分。
- 好的，我会根据您提供的对话内容，以一个大一新生的语气总结今天的任务三学习笔记，确保内容贴合您的学习经历，并对以后复习有帮助。以下是基于对话的总结：

学习内容总结

- **任务目标**：任务三的重点是通过改进CNN模型来优化性能，今天成功运行了`cifar10_cnn_with_dropout_bn.py`文件，完成了10个epoch的训练，学习了`Dropout`和`BatchNorm`的作用，并对比了改进前后的结果。
- **关键知识点**：
    - **Dropout的作用**：Dropout是一种正则化技术，通过随机丢弃神经元，防止模型过拟合。复习要点：
        - Dropout概率（如0.5）越高，训练时丢弃的神经元越多，模型越不容易过拟合，但训练准确率会降低。
        - 今天的结果显示，Dropout显著减少了过拟合，训练和测试准确率差距从25.84%缩小到1.78%。
    - **BatchNorm的作用**：BatchNorm（批归一化）通过对每一小批数据进行标准化，稳定训练过程，使损失和准确率变化更平滑。复习要点：
        - BatchNorm有助于模型更快收敛，减少训练过程中的波动。
    - **训练结果对比**：通过对比添加Dropout和BatchNorm前后的结果，理解了正则化的效果。复习时可以看这两个表格：
        - **改进前（无Dropout和BatchNorm）**：

```
| Epoch | Train Loss | Train Acc (%) | Test Acc (%) |
|-------|------------|---------------|--------------|
| 1     | 0.273      | 49.02         | 60.36        |
| 2     | 0.219      | 64.32         | 67.27        |
| 3     | 0.187      | 70.99         | 68.90        |
| 4     | 0.154      | 76.08         | 69.27        |
| 5     | 0.128      | 80.82         | 71.15        |
| 6     | 0.103      | 85.39         | 71.94        |
| 7     | 0.079      | 89.36         | 72.18        |
| 8     | 0.061      | 92.34         | 71.15        |
| 9     | 0.043      | 94.67         | 70.74        |
| 10    | 0.032      | 95.98         | 70.14        |
```

        - **改进后（添加Dropout和BatchNorm）**：

```
| Epoch | Train Loss | Train Acc (%) | Test Acc (%) |
|-------|------------|---------------|--------------|
| 1     | 0.983      | 43.91         | 56.98        |
| 2     | 0.867      | 56.41         | 61.32        |
| 3     | 0.796      | 61.21         | 65.13        |
| 4     | 0.745      | 64.14         | 68.70        |
| 5     | 0.713      | 66.02         | 69.26        |
| 6     | 0.684      | 67.81         | 69.82        |
| 7     | 0.643      | 69.35         | 70.82        |
| 8     | 0.627      | 70.52         | 70.40        |
| 9     | 0.597      | 71.68         | 70.71        |
| 10    | 0.579      | 72.87         | 71.09        |
```

分析要点：
  - 改进后训练损失较高（从0.983降到0.579），训练准确率较低（从43.91%升到72.87%），远低于改进前的95.98%，这是Dropout正则化的预期效果。
  - 测试准确率最终为71.09%，与改进前最佳值72.18%接近，但过拟合显著减少（训练和测试准确率差距很小）。
- **实践操作**：
  - 今天成功运行了`cifar10_cnn_with_dropout_bn.py`，完成了10个epoch训练，观察到Dropout和BatchNorm对过拟合的控制效果。
  - 对比了改进前后的训练和测试结果，记录了数据表格，保存了模型文件`cifar10_cnn_with_dropout_bn.pth`。
- **实践疑惑**：
  - 还有点困惑的是Dropout概率具体怎么调，0.3或0.2会不会更好，打算以后多实验几次。
- **心得体会**：
  - 今天学下来，感觉Dropout和BatchNorm真的很有效，过拟合问题明显改善，虽然测试准确率没提高太多，但模型更稳定了。
  - 跑通程序看到结果对比，觉得深度学习的调参挺有意思的。希望多学多练，慢慢搞懂这些优化方法。

学习内容总结

- **任务目标**：任务三的重点是通过调整Dropout概率来优化CNN模型性能，今天分析了Dropout概率为0.45的训练结果，并与之前0.3、0.5和0.6的结果进行对比，寻找最佳参数。
- **关键知识点**：
  - **Dropout=0.45的训练结果**：训练了10个epoch，记录了训练损失、训练准确率和测试准确率的变化。复习时可以看这个分析：
    - 训练损失：从0.952（Epoch 1）降到0.510（Epoch 10），模型在训练数据上逐步优化。
    - 训练准确率：从45.37%升到75.75%，学习能力不错。
    - 测试准确率：从58.76%升到73.02%（Epoch 10），峰值为73.02%。
    - 训练准确率（75.75%）和测试准确率（73.02%）差距小（2.73%），过拟合控制得很好。
  - **与不同Dropout概率的对比**：通过对比Dropout=0.3、0.45、0.5和0.6的结果，理解了Dropout概率对模型的影响。复习时可以看这个表格：

```
| Dropout 概率 | 最终训练准确率 | 最终测试准确率 | 训练-测试准确率差 | 过拟合程度 |
```

```
|--------------|----------------|----------------|--------------------
|------------------|
| 0.3          | 80.81%         | 72.89%         | 7.92%             |
较明显           |
| 0.45         | 75.75%         | 73.02%         | 2.73%             |
较小             |
| 0.5          | 72.87%         | 71.09%         | 1.78%             |
较小             |
| 0.6          | 68.73%         | 70.87%         | -2.14%            |
几乎无过拟合     |
```

分析要点：

- Dropout=0.45比0.3有过拟合更少（差距2.73% vs 7.92%），测试准确率略高（73.02% vs 72.89%）。
- 比Dropout=0.5，0.45的训练和测试准确率更高（75.75% vs 72.87%，73.02% vs 71.09%），学习能力更好。
- 比Dropout=0.6，0.45的准确率更高（75.75% vs 68.73%，73.02% vs 70.87%），避免了欠拟合。
- 综合来看，Dropout=0.45在过拟合控制和学习能力之间达到最佳平衡，测试准确率73.02%是目前最高的。

- **实践操作**：
  - 今天分析了Dropout=0.45的训练日志，观察到测试准确率达到73.02%，是目前最高，验证了参数调整的效果。
  - 通过对比不同Dropout概率的结果，理解了如何平衡过拟合和学习能力。。
- **心得体会**：
  - 今天学下来，感觉Dropout概率调整真的很重要，0.45的效果最好，既控制了过拟合，又保持了学习能力，测试准确率达到73.02%。
  - 看到不同参数的对比结果，觉得深度学习调参挺有意思的。希望多实验多总结，慢慢掌握这些技巧。

学习内容总结

- **任务目标**：任务三的重点是通过优化方法提升CNN模型在CIFAR-10数据集上的性能，今天学习了学习率调度的原理和应用，了解如何动态调整学习率以帮助模型更好地收敛。
- **关键知识点**：
  - **学习率调度的概念**：学习率是优化算法中控制参数更新步长的超参数，学习率调度是指在训练过程中动态调整学习率。复习要点：
    - **为什么要调度**：初期高学习率帮助快速学习，后期低学习率精细调整，避免震荡，提高性能。
    - **常见策略**：
      - **Step Decay（阶梯衰减）**：每隔固定epoch数学习率减半。
      - **Exponential Decay（指数衰减）**：学习率随epoch指数下降。
      - **Cosine Annealing（余弦退火）**：学习率按余弦曲线下降。
      - **Warmup（预热）**：初期逐渐增加学习率，再衰减。
  - **PyTorch中实现学习率调度**：以`StepLR`为例，学习了如何在代码中应用学习率调度。复习时可以看这个代码片段：

```
from torch.optim.lr_scheduler import StepLR
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
scheduler = StepLR(optimizer, step_size=3, gamma=0.5)  # 每3个epoch学习率减半
for epoch in range(10):
    # 训练代码...
    scheduler.step()  # 更新学习率
```

代码解释：
  - `StepLR(optimizer, step_size=3, gamma=0.5)`：每3个epoch将学习率乘以0.5（减半）。
  - `scheduler.step()`：每个epoch结束时调用，更新学习率。
  - 效果：初始学习率0.01，第4-6个epoch变为0.005，第7-9个epoch变为0.0025，以此类推。
- **在CIFAR-10模型中应用**：学习了如何将学习率调度集成到训练代码中。复习要点：
  - 将`scheduler.step()`放在每个epoch最后，确保学习率更新。
  - 可以打印当前学习率（如`optimizer.param_groups[0]["lr"]`）确认调度生效。
- **其他调度策略**：PyTorch还提供了多种调度器，适用于不同场景。复习时记得：
  - `ExponentialLR`：指数衰减，每epoch学习率乘以固定因子（如0.9）。
  - `CosineAnnealingLR`：余弦退火，学习率按余弦曲线变化。
  - `ReduceLROnPlateau`：根据指标（如测试损失）动态调整学习率。
- **应用使用**：对于我的CIFAR-10模型，用`StepLR`（如`step_size=3, gamma=0.5`），并增加训练轮数到15或20个epoch，看测试准确率能否超73.02%。

- **实践操作**：
  - 没有直接跑代码，但详细学习了学习率调度的原理和代码实现，理解了如何在训练中动态调整学习率。
  - 结合之前的训练结果（测试准确率在第9-10个epoch仍在提升），认识到学习率调度可能帮助模型进一步收敛。
- **遇到的问题**：
  - 目前没有实际运行代码，所以没有新问题，但对学习率调度的具体效果还有点模糊，比如`StepLR`和`CosineAnnealingLR`哪个更适合我的模型，打算以后实验对比。
- **心得体会**：
  - 今天学下来，感觉学习率调度是个很重要的优化技巧，通过动态调整学习率，可以让模型训练更高效，后期更精确。
  - 虽然还没实际应用，但看到代码例子，觉得挺有意思的。希望多学多练，慢慢把这些优化方法用起来！

学习内容总结

- **任务目标**：任务三的重点是通过优化方法提升CNN模型性能，今天分析了Dropout=0.45并使用`StepLR`学习率调度的训练结果（15个epoch），并学习了解决OpenMP错误和进一步优化的方法。
- **关键知识点**：
  - **训练结果分析**：训练了15个epoch，记录了训练损失、训练准确率和测试准确率的变化。复习时可以看这个分析：
    - 训练损失：从0.975（Epoch 1）降到0.447（Epoch 15），模型持续优化。
    - 训练准确率：从44.20%升到78.25%，学习能力不错。
    - 测试准确率：从58.51%升到73.79%（Epoch 15），峰值为73.84%（Epoch 14）。

        - 训练准确率（78.25%）和测试准确率（73.79%）差距为4.46%，过拟合有所控制但仍存在。
    - **学习率调度效果**：学习率按`StepLR`计划在第3、6、9、12个epoch后减半，从0.001降到0.000031。复习要点：
        - 测试准确率在学习率降低后仍有小幅提升（第10个epoch后从73.05%到73.84%），说明调度有效。
        - 第14-15个epoch测试准确率波动小，可能接近收敛或学习率过低导致更新不足。
- **实践操作**：
    - 今天分析了Dropout=0.45并使用学习率调度的训练日志，测试准确率从73.02%提升到73.84%，验证了学习率调度的效果。
    - 学习了OpenMP错误的解决方法，准备在下次运行时尝试设置环境变量。
- **遇到的问题**：
    - 对学习率后期过低导致更新不足有点困惑，打算试试调整`gamma`或换其他调度策略。
- **心得体会**：
    - 今天学下来，感觉学习率调度真的有用，测试准确率小幅提升到73.84%，而且了解了如何解决OpenMP错误，挺有收获的。
    - 看到优化建议，觉得深度学习调参和模型改进的空间很大。希望多实验多总结，慢慢提高模型性能！

学习内容总结

- **任务目标**：任务三的重点是通过优化CNN模型提升在CIFAR-10数据集上的性能，今天分析了两次训练结果（增加一个卷积层，Dropout分别为0.45和0.5），并学习了解决OpenMP错误和进一步优化的方法。
- **关键知识点**：
    - **第一次训练结果（Dropout=0.45，增加一个卷积层）**：训练了15个epoch，复习时可以看这个分析：
        - 训练损失：从0.878（Epoch 1）降到0.303（Epoch 15），模型优化明显。
        - 训练准确率：从47.25%升到85.84%，学习能力增强。
        - 测试准确率：从62.88%升到77.67%（峰值，第13个epoch），最终为77.07%（Epoch 15）。
        - 训练准确率（85.84%）和测试准确率（77.07%）差距为8.77%，过拟合较明显。
    - **第二次训练结果（Dropout=0.5，增加一个卷积层）**：同样训练15个epoch，复习时记得：
        - 训练损失：从0.967降到0.329，略高于第一次。
        - 训练准确率：从44.25%升到83.66%，略低于第一次。
        - 测试准确率：从58.62%升到77.27%（峰值，第15个epoch）。
        - 训练准确率（83.66%）和测试准确率（77.27%）差距为6.39%，过拟合减少。
    - **综合分析**：对比两次训练结果，复习要点：
        - 增加卷积层后，测试准确率从之前的74.56%提升到77.67%，说明更深的模型提取特征能力更强。
        - Dropout从0.45增加到0.5，过拟合减少（差距从8.77%降到6.39%），但测试准确率略降（77.67%到77.27%）。
        - 学习率调度（`gamma=0.7`）在后期仍推动测试准确率提升。
- **实践操作**：
    - 今天分析了两次训练结果，观察到增加卷积层显著提升测试准确率（峰值77.67%），Dropout=0.5减少了过拟合。
    - 学习了OpenMP错误的解决方法，准备下次运行时设置环境变量。
- **遇到的问题**：

    - 对过拟合问题还有点困惑，虽然Dropout=0.5减少了差距，但测试准确率略降，打算试试数据增强和调整Dropout率。
- **心得体会**：
    - 学下来，感觉模型结构改进和Dropout调整真的很重要，测试准确率提升到77.67%，而且过拟合问题通过提高Dropout率有所缓解。
    - 看到优化建议，觉得深度学习调参和改进的空间很大。希望多实验多总结，慢慢提高模型性能。

学习内容总结

- **任务目标**：任务三的重点是通过优化CNN模型提升性能，今天分析了Dropout=0.48并增加一个卷积层的训练结果（15个epoch），并学习了进一步优化的方法。
- **关键知识点**：
    - **训练结果分析**：训练了15个epoch，记录了训练损失、训练准确率和测试准确率的变化。复习时可以看这个分析：
        - 训练损失：从0.903（Epoch 1）降到0.320（Epoch 15），模型持续优化。
        - 训练准确率：从46.56%升到84.83%，学习能力增强。
        - 测试准确率：从57.52%升到78.17%（峰值，第15个epoch），高于之前Dropout=0.45（77.67%）和0.5（77.27%）。
        - 训练准确率（84.83%）和测试准确率（78.17%）差距为6.66%，介于Dropout=0.45（8.77%）和0.5（6.39%）之间。
    - **学习率调度效果**：学习率按`StepLR`计划（`gamma=0.7`）在第3、6、9、12个epoch后减少，从0.001降到0.000168。复习要点：
        - 测试准确率在后期持续提升（第13-15个epoch从76.98%到78.17%），说明学习率调度和Dropout调整有效。
    - **观察结果**：Dropout=0.48是更好的平衡点，测试准确率达到78.17%，过拟合控制也比0.45好，但仍有差距（6.66%）。
- **实践操作**：
    - 今天分析了Dropout=0.48的训练日志，测试准确率提升到78.17%，验证了参数调整的效果。
    - 观察到Dropout=0.48在控制过拟合和提升准确率之间达到了更好的平衡。
- **遇到的问题**：
    - 目前没有新问题，但过拟合仍有差距（6.66%），打算试试数据增强等方法进一步减少。
- **心得体会**：
    - 今天学下来，感觉Dropout调整真的很重要，0.48的效果更好，测试准确率达到78.17%，而且过拟合问题有所改善。
    - 看到各种优化建议，觉得深度学习调参和模型改进的空间很大。希望多实验多总结，慢慢提高模型性能。

学习内容总结

- **任务目标**：任务三的重点是通过优化CNN模型提升性能，今天分析了Dropout=0.48、增加一个卷积层并加入数据增强的训练结果（15个epoch），并学习了测试准确率下降的原因和改进建议。
- **关键知识点**：
    - **训练结果分析**：训练了15个epoch，记录了训练损失、训练准确率和测试准确率的变化。复习时可以看这个分析：
        - 训练损失：从1.067（Epoch 1）降到0.570（Epoch 15），模型持续优化。
        - 训练准确率：从39.25%升到72.59%，低于之前的84.83%。
        - 测试准确率：从52.32%升到76.48%（峰值，第15个epoch），低于之前的78.17%。

        - 训练准确率（72.59%）和测试准确率（76.48%）差距为-3.89%，过拟合问题消除，甚至有轻微欠拟合。
    - **数据增强效果**：加入`RandomCrop`和`RandomHorizontalFlip`后，复习要点：
        - 训练准确率显著下降（从84.83%到72.59%），测试准确率略降（从78.17%到76.48%）。
        - 训练-测试准确率差距从6.66%变为负值，说明数据增强控制了过拟合，但可能过于强烈，导致学习能力受限。
    - **原因分析**：测试准确率下降的原因，复习时记得：
        - 数据增强（`RandomCrop`和`RandomHorizontalFlip`）增加训练难度，模型在15个epoch内未适应。
        - 训练轮数不足，15个epoch可能不够让模型收敛。
        - 学习率调度（`gamma=0.7`）后期下降过快，更新步长不足。
- **实践操作**：
    - 分析了加入数据增强后的训练日志，测试准确率下降到76.48%，了解了数据增强对模型的影响。
    - 认识到数据增强控制了过拟合，但也增加了学习难度，导致准确率下降。
- **遇到的问题**：
    - 测试准确率从78.17%下降到76.48%，主要是数据增强增加难度，训练轮数不足。
- **心得体会**：
    - 学下来，感觉数据增强是个双刃剑，虽然解决了过拟合，但也让模型学习更难，测试准确率下降到76.48%。
    - 看到改进建议，觉得深度学习优化需要综合考虑训练时间、参数调整和模型结构。希望多实验多总结，慢慢提高模型性能！

学习内容总结

- **任务目标**：任务三的重点是通过优化CNN模型提升在CIFAR-10数据集上的性能，今天学习了如何通过代码实现增加训练轮数到25个epoch并加入早停机制，同时复习了模型结构和数据增强的设置。
- **关键知识点**：
    - **代码实现内容**：复习时可以重点看以下部分：
        - **增加训练轮数**：将`num_epochs`从15增加到25，给予模型更多时间适应数据增强带来的多样性。
        - **早停机制**：设置`patience=5`，如果连续5个epoch测试准确率没有提升，则停止训练；每次提升时保存最佳模型到`best_model.pth`。
        - **模型保存**：最佳模型保存为`best_model.pth`，最终模型保存为`cifar10_cnn_with_dropout_bn.pth`。
        - **绘制曲线**：训练结束后，绘制并保存损失和准确率曲线到`training_curves_with_dropout_bn.png`。
    - **模型结构**：复习时记得当前CNN模型配置：
        - 3个卷积层（通道数：3->16->32->64），包含BatchNorm和Dropout（0.48）。
        - 全连接层：`64*4*4->512->256->10`，两层Dropout（0.48）。
    - **数据增强**：训练数据增强包括`RandomCrop`和`RandomHorizontalFlip`，测试数据无增强，仅归一化。
    - **学习率调度**：使用`StepLR`，每3个epoch学习率减少到70%（`gamma=0.7`）。
    - **预期效果**：复习时记得代码的优化目标：
        - 增加到25个epoch，模型有更多机会学习，训练和测试准确率有望提升。
        - 早停机制避免资源浪费，确保在性能不再提升时停止。
        - 测试准确率有望超过之前的76.48%，甚至接近或超过未加数据增强时的78.17%。
- **实践操作**：

    - 今天学习了增加训练轮数和早停机制的具体代码，理解了如何通过代码控制训练过程。
    - 复习了模型结构、数据增强和学习率调度的设置，准备下次运行时尝试。
- **遇到的问题**：
    - 对早停机制的具体效果还不太清楚，打算运行代码后观察是否能提升测试准确率。
- **心得体会**：
    - 今天学下来，感觉增加训练轮数和加入早停机制是优化训练过程的好方法，既能提高准确率，又能节省资源。
    - 看到代码实现，觉得深度学习不仅是调参，还有很多工程上的细节需要注意。希望多实践多总结，慢慢掌握这些技巧！

测试准确率高于训练准确率是一种不常见的现象，但在某些情况下确实会发生，尤其是在您的 CIFAR-10 CNN 模型训练中观察到的结果（训练准确率 74.67%，测试准确率 77.61%，差距为 -2.94%）。以下是导致这种情况的具体原因分析：

1. 数据增强仅应用于训练数据

- **原因**：您在训练数据上使用了数据增强（`RandomCrop` 和 `RandomHorizontalFlip`），而测试数据未应用这些增强操作。数据增强会增加训练数据的多样性和难度，使得模型在训练时面对的是“更难”的数据分布（例如，随机裁剪和翻转会改变图像的局部特征），从而导致训练准确率较低。而测试数据是未经增强的“标准”数据，模型可能更容易识别这些未变形的数据，导致测试准确率较高。
- **影响**：数据增强的主要目的是提高模型的泛化能力，防止过拟合。虽然它成功地减少了过拟合（甚至导致训练准确率低于测试准确率），但也表明模型在训练数据上的学习可能不够充分。

2. Dropout 的影响

- **原因**：您的模型中使用了 Dropout（比例为 0.48），Dropout 在训练时会随机丢弃一部分神经元以防止过拟合，但这也使得模型在训练阶段的表现受到限制（即训练准确率较低）。而在测试阶段，Dropout 不会被应用，模型会使用完整的网络权重，因此可能在测试数据上表现更好。
- **影响**：Dropout 是一种正则化技术，通常会降低训练准确率以提高泛化能力。当测试准确率高于训练准确率时，表明 Dropout 可能过于强烈，限制了模型在训练数据上的学习能力。

3. 训练数据和测试数据的分布差异

- **原因**：CIFAR-10 数据集的训练集和测试集可能存在细微的分布差异。尽管数据集是随机划分的，但某些类别的图像在测试集中可能更容易被分类（例如，测试集中的图像可能具有更清晰的背景或更明显的特征）。此外，由于数据增强改变了训练数据的分布（使其更难），模型在未经增强的测试数据上可能表现得更好。
- **影响**：这种分布差异通常较小，但在数据增强和正则化（如 Dropout）的共同作用下，可能会导致测试准确率高于训练准确率。

4. 模型未充分收敛（训练不足）

- **原因**：尽管训练轮数增加到了 25 个 epoch，但由于数据增强增加了学习难度，模型可能尚未在训练数据上充分收敛。训练准确率（74.67%）相比之前的实验（未加数据增强时为 84.83%）明显较低，表明模型在训练数据上的学习能力被限制，可能需要更多训练轮数或调整学习率调度策略。
- **影响**：训练不足会导致模型无法完全拟合训练数据，而测试数据由于分布相对简单（无增强），模型可能表现得更好。

5. 正则化效果过于强烈

- **原因**：除了 Dropout 外，数据增强和 BatchNorm 等正则化技术也在起作用。这些技术共同作用可能过度限制了模型对训练数据的拟合能力，导致训练准确率被压低。而测试数据由于没有这些正则化限制（例如无数据增强、无 Dropout），模型表现反而更好。
- **影响**：过强的正则化会导致轻微的欠拟合，表现为训练准确率低于测试准确率。